

## Constructing a Surrogate

### Note

This section is based on chapter 2 in @Forr08a.

**Definition 0.1** (Black Box Problem). We are trying to learn a mapping that converts the vector  $\vec{x}$  into a scalar output  $y$ , i.e., we are trying to learn a function

$$y = f(x).$$

If function is hidden (“lives in a black box”), so that the physics of the problem is not known, the problem is called a black box problem.

This black box could take the form of either a physical or computer experiment, for example, a finite element code, which calculates the maximum stress ( $\sigma$ ) for given product dimensions ( $\vec{x}$ ).

**Definition 0.2** (Generic Solution). The generic solution method is to collect the output values  $y^{(1)}, y^{(2)}, \dots, y^{(n)}$  that result from a set of inputs  $\vec{x}^{(1)}, \vec{x}^{(2)}, \dots, \vec{x}^{(n)}$  and find a best guess  $\hat{f}(\vec{x})$  for the black box mapping  $f$ , based on these known observations.

### Stage One: Preparing the Data and Choosing a Modelling Approach

The first step is the identification, through a small number of observations, of the inputs that have a significant impact on  $f$ ; that is the determination of the shortest design variable vector  $\vec{x} = \{x_1, x_2, \dots, x_k\}^T$  that, by sweeping the ranges of all of its variables, can still elicit most of the behavior the black box is capable of. The ranges of the various design variables also have to be established at this stage.

The second step is to recruit  $n$  of these  $k$ -vectors into a list

$$X = \{\vec{x}^{(1)}, \vec{x}^{(2)}, \dots, \vec{x}^{(n)}\}^T,$$

where each  $\vec{x}^{(i)}$  is a  $k$ -vector. The corresponding responses are collected in a vector such that this represents the design space as thoroughly as possible.

In the surrogate modeling process, the number of samples  $n$  is often limited, as it is constrained by the computational cost (money and/or time) associated with obtaining each observation.

It is advisable to scale  $\vec{x}$  at this stage into the unit cube  $[0, 1]^k$ , a step that can simplify the subsequent mathematics and prevent multidimensional scaling issues.

We now focus on the attempt to learn  $f$  through data pairs

$$\{(\vec{x}^{(1)}, y^{(1)}), (\vec{x}^{(2)}, y^{(2)}), \dots, (\vec{x}^{(n)}, y^{(n)})\}.$$

This supervised learning process essentially involves searching across the space of possible functions  $\hat{f}$  that would replicate observations of  $f$ . This space of functions is infinite. Any number of hypersurfaces could be drawn to pass through or near the known observations, accounting for experimental error. However, most of these would generalize poorly; they would be practically useless at predicting responses at new sites, which is the ultimate goal.

**Example 0.1** (The Needle(s) in the Haystack Function). An extreme example is the ‘needle(s) in the haystack’ function:

$$f(x) = \begin{cases} y^{(1)}, & \text{if } x = \vec{x}^{(1)} \\ y^{(2)}, & \text{if } x = \vec{x}^{(2)} \\ \vdots \\ y^{(n)}, & \text{if } x = \vec{x}^{(n)} \\ 0, & \text{otherwise.} \end{cases}$$

While this predictor reproduces all training data, it seems counter-intuitive and unsettling to predict 0 everywhere else for most engineering functions. Although there is a small chance that the function genuinely resembles the equation above and we sampled exactly where the needles are, it is highly unlikely.

There are countless other configurations, perhaps less contrived, that still generalize poorly. This suggests a need for systematic means to filter out nonsensical predictors. In our approach, we embed the structure of  $f$  into the model selection algorithm and search over its parameters to fine-tune the approximation to observations. For instance, consider one of the simplest models,

$$f(x, \vec{w}) = \vec{w}^T \vec{x} + v. \tag{1}$$

Learning  $f$  with this model implies that its structure—a hyperplane—is predetermined, and the fitting process involves finding the  $k + 1$  parameters (the slope vector  $\vec{w}$  and the intercept  $v$ ) that best fit the data. This will be accomplished in Stage Two.

Complicating this further is the noise present in observed responses (we assume design vectors  $\vec{x}$  are not corrupted). Here, we focus on learning from such data, which sometimes risks overfitting.

**Definition 0.3** (Overfitting). Overfitting occurs when the model becomes too flexible and captures not only the underlying trend but also the noise in the data.

In the surrogate modeling process, the second stage as described in Section , addresses this issue of complexity control by estimating the parameters of the fixed structure model. However, foresight is necessary even at the model type selection stage.

Model selection often involves physics-based considerations, where the modeling technique is chosen based on expected underlying responses.

**Example 0.2** (Model Selection). Modeling stress in an elastically deformed solid due to small strains may justify using a simple linear approximation. Without insights into the physics, and if one fails to account for the simplicity of the data, a more complex and excessively flexible model may be incorrectly chosen. Although parameter estimation might still adjust the approximation to become linear, an opportunity to develop a simpler and robust model may be lost.

- Simple linear (or polynomial) models, despite their lack of flexibility, have advantages like applicability in further symbolic computations.
- Conversely, if we incorrectly assume a quadratic process when multiple peaks and troughs exist, the parameter estimation stage will not compensate for an unsuitable model choice. A quadratic model is too rigid to fit a multimodal function, regardless of parameter adjustments.

## Stage Two: Parameter Estimation and Training

Assuming that Stage One helped identify the  $k$  critical design variables, acquire the learning data set, and select a generic model structure  $f(\vec{x}, \vec{w})$ , the task now is to estimate parameters  $\vec{w}$  to ensure the model fits the data optimally. Among several estimation criteria, we will discuss two methods here.

**Definition 0.4** (Maximum Likelihood Estimation). Given a set of parameters  $\vec{w}$ , the model  $f(\vec{x}, \vec{w})$  allows computation of the probability of the data set

$$\{(\vec{x}^{(1)}, y^{(1)} \pm \epsilon), (\vec{x}^{(2)}, y^{(2)} \pm \epsilon), \dots, (\vec{x}^{(n)}, y^{(n)} \pm \epsilon)\}$$

resulting from  $f$  (where  $\epsilon$  is a small error margin around each data point).

### **i** Maximum Likelihood Estimation

**?@sec-max-likelihood** presents a more detailed discussion of the maximum likelihood estimation (MLE) method.

Taking **?@eq-likelihood-mvn** and assuming errors  $\epsilon$  are independently and normally distributed with standard deviation  $\sigma$ , the probability of the data set is given by:

$$P = \frac{1}{(2\pi\sigma^2)^{n/2}} \exp \left[ -\frac{1}{2\sigma^2} \sum_{i=1}^n (y^{(i)} - f(\vec{x}^{(i)}, \vec{w}))^2 \epsilon \right].$$

Intuitively, this is equivalent to the likelihood of the parameters given the data. Accepting this intuitive relationship as a mathematical one aids in model parameter estimation. This is achieved by maximizing the likelihood or, more conveniently, minimizing the negative of its natural logarithm:

$$\min_{\vec{w}} \sum_{i=1}^n \frac{[y^{(i)} - f(\vec{x}^{(i)}, \vec{w})]^2}{2\sigma^2} + \frac{n}{2} \ln \epsilon. \quad (2)$$

If we assume  $\sigma$  and  $\epsilon$  are constants, Equation 2 simplifies to the well-known least squares criterion:

$$\min_{\vec{w}} \sum_{i=1}^n [y^{(i)} - f(\vec{x}^{(i)}, \vec{w})]^2.$$

Cross-validation is another method used to estimate model performance.

**Definition 0.5** (Cross-Validation). Cross-validation splits the data randomly into  $q$  roughly equal subsets, and then cyclically removing each subset and fitting the model to the remaining  $q - 1$  subsets. A loss function  $L$  is then computed to measure the error between the predictor and the withheld subset for each iteration, with contributions summed over all  $q$  iterations. More formally, if a mapping  $\theta : \{1, \dots, n\} \rightarrow \{1, \dots, q\}$  describes the allocation of the  $n$  training points to one of the  $q$  subsets and  $f^{(-\theta(i))}(\vec{x})$  is the predicted value by removing the subset  $\theta(i)$  (i.e., the subset where observation  $i$  belongs), the cross-validation measure, used as an estimate of prediction error, is:

$$CV = \frac{1}{n} \sum_{i=1}^n L(y^{(i)}, f^{(-\theta(i))}(\vec{x}^{(i)})). \quad (3)$$

Introducing the squared error as the loss function and considering our generic model  $f$  still dependent on undetermined parameters, we write Equation 3 as:

$$CV = \frac{1}{n} \sum_{i=1}^n [y^{(i)} - f^{(-\theta(i))}(\vec{x}^{(i)})]^2. \quad (4)$$

The extent to which Equation 4 is an unbiased estimator of true risk depends on  $q$ . It is shown that if  $q = n$ , the leave-one-out cross-validation (LOOCV) measure is almost unbiased. However, LOOCV can have high variance because subsets are very similar. @Hast17a) suggest using

compromise values like  $q = 5$  or  $q = 10$ . Using fewer subsets also reduces the computational cost of the cross-validation process, see also @arlot2010 and @Koha95a.

### Stage Three: Model Testing

If there is a sufficient amount of observational data, a random subset should be set aside initially for model testing. @Hast17a recommend setting aside approximately  $0.25n$  of  $\vec{x} \rightarrow y$  pairs for testing purposes. These observations must remain untouched during Stages One and Two, as their sole purpose is to evaluate the testing error—the difference between true and approximated function values at the test sites—once the model has been built. Interestingly, if the main goal is to construct an initial surrogate for seeding a global refinement criterion-based strategy (as discussed in Section 3.2 in @Forr08a), the model testing phase might be skipped.

It is noted that, ideally, parameter estimation (Stage Two) should also rely on a separate subset. However, observational data is rarely abundant enough to afford this luxury (if the function is cheap to evaluate and evaluation sites are selectable, a surrogate model might not be necessary).

When data are available for model testing and the primary objective is a globally accurate model, using either a root mean square error (RMSE) metric or the correlation coefficient ( $r^2$ ) is recommended. To test the model, a test data set of size  $n_t$  is used alongside predictions at the corresponding locations to calculate these metrics.

The RMSE is defined as follows:

**Definition 0.6** (Root Mean Square Error (RMSE)).

$$\text{RMSE} = \sqrt{\frac{1}{n_t} \sum_{i=1}^{n_t} (y^{(i)} - \hat{y}^{(i)})^2},$$

Ideally, the RMSE should be minimized, acknowledging its limitation by errors in the objective function  $f$  calculation. If the error level is known, like a standard deviation, the aim might be to achieve an RMSE within this value. Often, the target is an RMSE within a specific percentage of the observed data’s objective value range.

The squared correlation coefficient  $r$ , see ?@eq-pears-corr, between the observed  $y$  and predicted  $\hat{y}$  values can be computed as:

$$r^2 = \left( \frac{\text{cov}(y, \hat{y})}{\sqrt{\text{var}(y)\text{var}(\hat{y})}} \right)^2, \quad (5)$$

Equation 5 and can be expanded as:

$$r^2 = \left( \frac{n_t \sum_{i=1}^{n_t} y^{(i)} \hat{y}^{(i)} - \sum_{i=1}^{n_t} y^{(i)} \sum_{i=1}^{n_t} \hat{y}^{(i)}}{\sqrt{\left( n_t \sum_{i=1}^{n_t} (y^{(i)})^2 - \left( \sum_{i=1}^{n_t} y^{(i)} \right)^2 \right) \left( n_t \sum_{i=1}^{n_t} (\hat{y}^{(i)})^2 - \left( \sum_{i=1}^{n_t} \hat{y}^{(i)} \right)^2 \right)}} \right)^2.$$

The correlation coefficient  $r^2$  does not require scaling the data sets and only compares landscape shapes, not values. An  $r^2 > 0.8$  typically indicates a surrogate with good predictive capability.

The methods outlined provide quantitative assessments of model accuracy, yet visual evaluations can also be insightful. In general, the RMSE will not reach zero but will stabilize around a low value. At this point, the surrogate model is saturated with data, and further additions do not enhance the model globally (though local improvements can occur at newly added points if using an interpolating model).

### The Tea and Sugar Analogy

@Forr08a illustrates this saturation point using a comparison with a cup of tea and sugar. The tea represents the surrogate model, and sugar represents data. Initially, the tea is unsweetened, and adding sugar increases its sweetness. Eventually, a saturation point is reached where no more sugar dissolves, and the tea cannot get any sweeter. Similarly, a more flexible model, like one with additional parameters or employing interpolation rather than regression, can increase the saturation point—akin to making a hotter cup of tea for dissolving more sugar.

### Jupyter Notebook

- The Jupyter-Notebook of this chapter is available on GitHub in the [Sequential Parameter Optimization Cookbook Repository](#)